

Chapter 1

Weighted Miles Documentation

1.1 Introduction

The `weighted_miles.py` script computes terrain-adjusted miles for a transmission line from project physical details. It reads miles per terrain type and terrain multipliers from `02_project_physical_details.yaml`, then returns weighted miles and an average terrain multiplier. These values are used downstream by `build_costs.py` and `insurance_costs.py` for cost-per-mile calculations.

1.2 Method Overview

1.2.1 Purpose

1. Load `02_project_physical_details.yaml` (via `path_config.YAMLS_DIR`).
2. For each terrain type, multiply `terrain_miles[terrain_type]` by `terrain_multipliers[terrain_type]` and sum to get weighted miles.
3. Compute average terrain multiplier as weighted miles divided by total (unweighted) miles.
4. Return (`weighted_miles`, `average_terrain_multiplier`); when run as `__main__`, print both values.

1.2.2 Calculation Approach

Data come from `project_physical_details["terrain"]`: `terrain_miles` is a dict mapping terrain type to miles; `terrain_multipliers` maps terrain type to a cost/mile multiplier.

$$\text{weighted_miles} = \sum_{\text{terrain_type}} \text{terrain_miles}[\text{terrain_type}] \times \text{terrain_multipliers}[\text{terrain_type}] \quad (1.1)$$

$$\text{total_miles} = \sum_{\text{terrain_type}} \text{terrain_miles}[\text{terrain_type}] \quad (1.2)$$

$$\text{average_terrain_multiplier} = \frac{\text{weighted_miles}}{\text{total_miles}} \quad (1.3)$$

1.2.3 Inputs and Outputs

Inputs:

- `02_project_physical_details.yaml` (path from `path_config.YAMLS_DIR`): under `terrain`, keys `terrain_miles` (dict: terrain type $\rightarrow$ miles) and `terrain_multipliers` (dict: terrain type $\rightarrow$ multiplier).

Outputs:

- Return value: (`weighted_miles: float`, `average_terrain_multiplier: float`). When the script is run standalone, both values are printed to stdout. The script does not write to the CSV or JSON output managers; downstream scripts (`build_costs.py`, `insurance_costs.py`) call `calculate_weighted_miles()` and use the result.

1.2.4 Integration with Other Scripts

- `path_config.py` — `YAMLS_DIR` for the physical details file path.
- `build_costs.py` — Imports `calculate_weighted_miles`, uses `weighted_miles` for conductor and structure cost per mile; passes `weighted_miles` and `average_terrain_multiplier` to the output manager.
- `insurance_costs.py` — Imports `calculate_weighted_miles` for use in its calculations.

1.3 Key Functions

1.3.1 `calculate_weighted_miles()`

Calculates weighted miles and average terrain multiplier from project physical details (YAML `02_project_physical_details.yaml`).

Parameters: None (reads from YAML path provided by `path_config`).

Returns: tuple: (`weighted_miles: float`, `average_terrain_multiplier: float`).

Key logic:

```

1 weighted_miles = 0
2 for terrain_type, terrain_miles in project_physical_details_df["
  terrain"][
3     "terrain_miles"
4 ].items():
5     if terrain_miles is None:
6         terrain_miles = 0
7     weighted_miles += (
8         terrain_miles
9         * project_physical_details_df["terrain"]["
10            terrain_multipliers"][
11                terrain_type
12            ]
13 )
14 average_terrain_multiplier = weighted_miles / sum(
15     project_physical_details_df["terrain"]["terrain_miles"].values()
16 )
17 return weighted_miles, average_terrain_multiplier

```

1.4 Example

Standalone run (prints weighted miles and average terrain multiplier):

```

1 cd scripts
2 python weighted_miles.py

```

Typical YAML structure under **terrain**:

```

1 terrain:
2   terrain_miles:
3     forested: 25
4     scrubbed_flat: 35
5     wetland: 5
6   terrain_multipliers:
7     forested: 2.25
8     scrubbed_flat: 1.0
9     wetland: 1.5

```

Example: weighted miles = $25 \times 2.25 + 35 \times 1.0 + 5 \times 1.5 = 56.25 + 35 + 7.5 = 98.75$; total miles = 65; average terrain multiplier = $98.75/65 \approx 1.52$.